

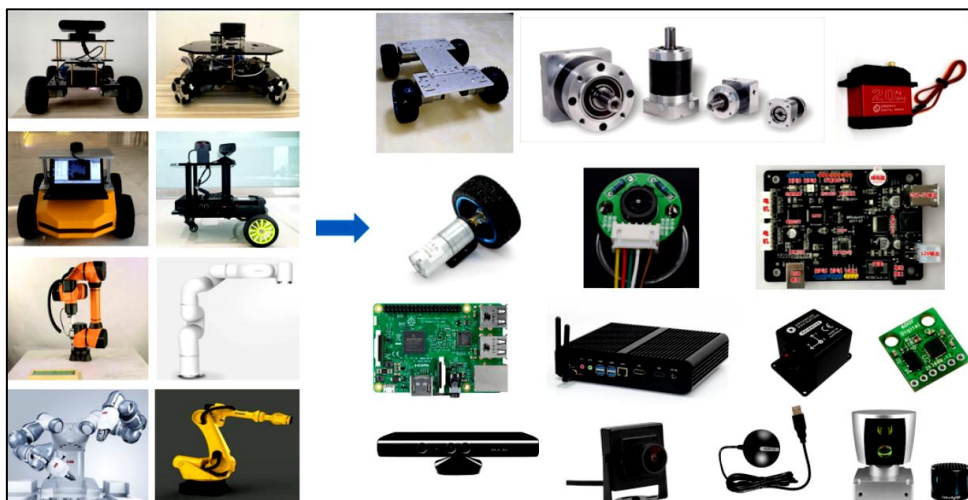
# Lesson 17 ROS2 URDF Robot Modeling Method

## 1. URDF Introduction

The modeling method in ROS is called URDF, which stands for United Robot Description Format. Not only can precisely describe the model of the robot itself, but also the environment surrounding the robot. The format of URDF model is XML.

## 2. Components of A Robot

During the process of modeling and describing robots, it's essential for us to familiarize with the components and parameters of the robot. For example, a robot typically consists of four major parts including hardware structure, drive system, sensor system, and control system. The robot. Whether it's mobile robot or desktop robotic arm, common robot on the market can be decomposed into these four main components.

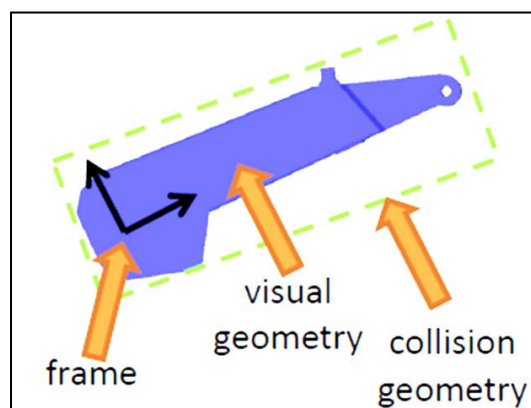


- Hardware structure refers to tangible components such as chassis, casing, motors, etc.
- The drive system consists of devices that enable the proper operation of these components. For example, motor drivers, power management systems and so on.
- The sensor system includes encoders on the motors, onboard IMU, mounted camera and lidar, and other devices, enabling the robot to perceive its own and the external environment.
- The control system serves as the primary platform in our development process, typically consisting of computing platforms such as Raspberry Pi, computers along with their operating systems and application software.

### 3. URDF Syntax

#### 3.1 Description of A Link

Tags are used to describe the appearance and physical properties of a specific rigid body part of a robot. Appearance includes dimensions, color and shape, while physical properties include mass, inertia matrix, collision parameters, etc.



Take the link of this robotic arm as example. Its links description is as follows:

```
<link name="link 4">

  <visual>

    <geometry>

      <mesh filename="link 4.stl"/>

    </geometry>

    <origin xyz="0 0 0" rpy="0 0 0" />

  </visual>

  <collision>

    <geometry>

      <cylinder length="0.5" radius="0.1"/>

    </geometry>

    <origin xyz="0 0 -0.05" rpy="0 0 0" />

  </collision>

</link>
```

The **<name>** attribute in the link tag represents the name of this link, which can be customized. When connecting joints to the link in the future, this name will be used.

The **<visual>** in the link tag is used to describe the appearance of the robot. For example:

- The **<geometry>** represents the geometric shape. It utilizes **<mesh>** to reference a pre-designed blue appearance created in 3D software, which is STL file. It appears consistent with the real robot.

- The <origin> represents the offset of the coordinates relative to the initial location. It includes translations in the direction of x, y and z, as well as rotations in roll, pitch, and yaw. If no offset is required, all values should be set to 0.

The second part <collision> describes the collision parameters. The contents inside are similar to <visual>, with <geometry> and <origin>. Though they seem alike, they actually serve distinct purposes.

- The <visual> section primarily describes the visual appearance of the robot, essentially the visual effects.
- The <collision> section focuses on the state of the robot during motion. For example, what kind of contact between robot and environment is regarded as collision.

### 3.2 Description of A Joint

In a robot model, the robot body needs to be connected through joints in order to generate relative motion.

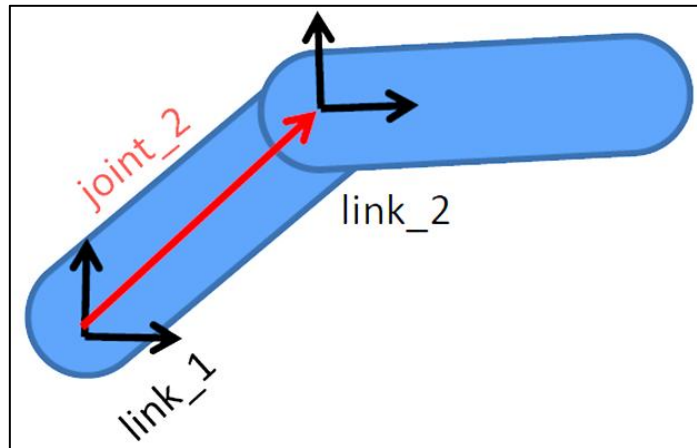
| Joint Type | Description                                                                                |
|------------|--------------------------------------------------------------------------------------------|
| continuous | A continuous joint allows infinite rotation around a single axis.                          |
| revolute   | A revolute joint is similar to continuous, with limits on the revolution angle.            |
| prismatic  | A prismatic joint is a sliding joint that moves along a single axis, with position limits. |

|          |                                                                                         |
|----------|-----------------------------------------------------------------------------------------|
| fixed    | A fixed joint is a special joint that does not allow any motion                         |
| floating | Floating joint allows both translation and rotation movements.                          |
| planar   | Planar joint allows translation or rotation along orthogonal directions within a plane. |

URDF defines six types of motion for joints.

The joint in URDF has six types of motion.

- 1) continuous: Describes rotational motion and allows infinite rotation around a specific axis. For example, the wheels of a car fall under this category.
- 2) revolute: Also a rotational joint, but unlike continuous joints, it has angle limitations. For example, the two links of a robotic arm fall under this type of motion.
- 3) Prismatic: A sliding joint that allows movement along a single axis, with position limits. Linear motors typically exhibit this type of motion.
- 4) Fixed: A fixed joint is the only type that does not allow any motion. Despite its immobility, it is frequently used. For instance, the link holding a camera mounted on a robot maintains a constant relative position, making it suitable for a fixed joint connection.
- 5) Floating: Floating joint allows free movement in space, typically with six degrees of freedom. This type of joint is less commonly used. The sixth type, planar, constrains motion to a specific plane, and both types are relatively less frequently used.



In the URDF model, each link is described using XML content, specifying details such as the name of the joint and its motion type.

```
<joint name="joint 2" type="revolute">  
  <parent link="link 1"/>  
  <child link="link 2"/>  
  <origin xyz="0.2 0.2 0" rpy="0 0 0"/>  
  <axis xyz="0 0 1"/>  
  <limit lower="-3.14" upper="3.14" velocity="1.0"/>  
</joint>
```

- parent tag: Describes the parent tag.
- child tag: Describes the child link, which moves relative to the parent link.
- origin: Represents the relationship between two link coordinate systems, depicted by the red vector in the diagram. It indicates how these two links should be installed together.
- Axis: Represents the unit vector of the joint motion axis. For example, if z equals 1, it indicates that the rotational motion occurs around the positive direction of the z-axis.

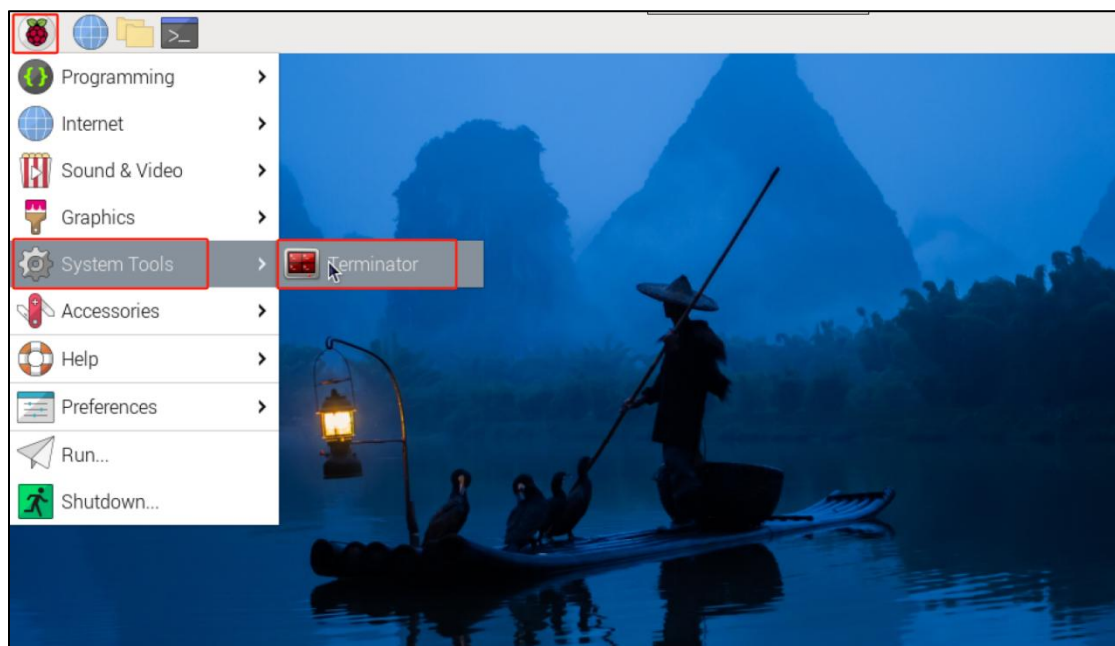
- Limit: Specifies constraints on the motion, such as minimum position, maximum position, and maximum velocity.

**Note:** In ROS, the default unit for translation is meters, and for rotation it's radians (not degrees). Therefore, in this context, 3.14 implies movement between -180 degrees and 180 degrees, linear velocity is in m/s, and angular velocity is in rad/s.

## 4. Complete Robot Model

### 4.1 Create Robot Model

1) Click  and select "System Tools → Terminator" in sequence.



2) Input the command "System Tools → Terminator" and press enter to switch to the src folder in hiwonder\_ws workspace.

```
ubuntu@raspberrypi:~$ cd hiwonder_ws/src/  
ubuntu@raspberrypi:~/hiwonder_ws/src$
```

3) Input the command "ros2 pkg create urdf\_demo --build-type

ament\_python --dependencies rclpy” and press Enter to creating a package named “**urdf\_demo**” with the dependency rclpy.

```
ubuntu@raspberrypi:~/hiwonder_ws/src$ ros2 pkg create urdf_demo --build-type ament_python --dependencies rclpy
```

4) Input the command “**cd urdf\_demo/**” to navigate to the **urdf\_demo** package.

```
ubuntu@raspberrypi:~/hiwonder_ws/src$ cd urdf_demo/  
ubuntu@raspberrypi:~/hiwonder_ws/src/urdf_demo$
```

5) Input the command “**mkdir urdf**” to create the urdf folder.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/urdf_demo$ mkdir urdf
```

6) Input the command “**cd urdf/**” to switch to the urdf folder.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/urdf_demo$ cd urdf/  
ubuntu@raspberrypi:~/hiwonder_ws/src/urdf_demo/urdf$
```

7) Input the command “**vim simple\_demo.urdf**” to open the file using VIM editor, copy and paste the program below. If you need to make modifications, you can press “i” to enter the insert mode. After you’ve finished the modifications, press “Esc” and enter “:wq” to save and exit.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/urdf_demo/urdf$ vim simple_demo.urdf
```

```
<?xml version="1.0"?>  
  
<robot name="materials">  
  
  <material name="blue">  
  
    <color rgba="0 0 0.8 1"/>  
  
  </material>
```

```
<material name="white">
```

```
  <color rgba="1 1 1 1"/>
```

```
</material>
```

```
<link name="base_link">
```

```
  <visual>
```

```
    <geometry>
```

```
      <cylinder length="0.6" radius="0.2"/>
```

```
    </geometry>
```

```
    <material name="blue"/>
```

```
  </visual>
```

```
</link>
```

```
<link name="right_leg">
```

```
  <visual>
```

```
    <geometry>
```

```
      <box size="0.6 0.1 0.2"/>
```

```
    </geometry>
```

```
    <origin rpy="0 1.57075 0" xyz="0 0 -0.3"/>
```

```
    <material name="white"/>
```

```
  </visual>
```

```
</link>
```

```
<joint name="base_to_right_leg" type="fixed">
```

```
  <parent link="base_link"/>
```

```
  <child link="right_leg"/>
```

```
  <origin xyz="0 -0.22 0.25"/>
```

```
</joint>
```

```
<link name="left_leg">
```

```
  <visual>
```

```
    <geometry>
```

```
      <box size="0.6 0.1 0.2"/>
```

```
    </geometry>
```

```
    <origin rpy="0 1.57075 0" xyz="0 0 -0.3"/>
```

```
    <material name="white"/>
```

```
  </visual>
```

```
</link>
```

```
<joint name="base_to_left_leg" type="fixed">
```

```
  <parent link="base_link"/>
```

```
  <child link="left_leg"/>
```

```
  <origin xyz="0 0.22 0.25"/>
```

```
</joint>
```

```
</robot>
```

```
<joint name="base_to_left_leg" type="fixed">
  <parent link="base_link"/>
  <child link="left_leg"/>
  <origin xyz="0 0.22 0.25"/>
</joint>

</robot>
```

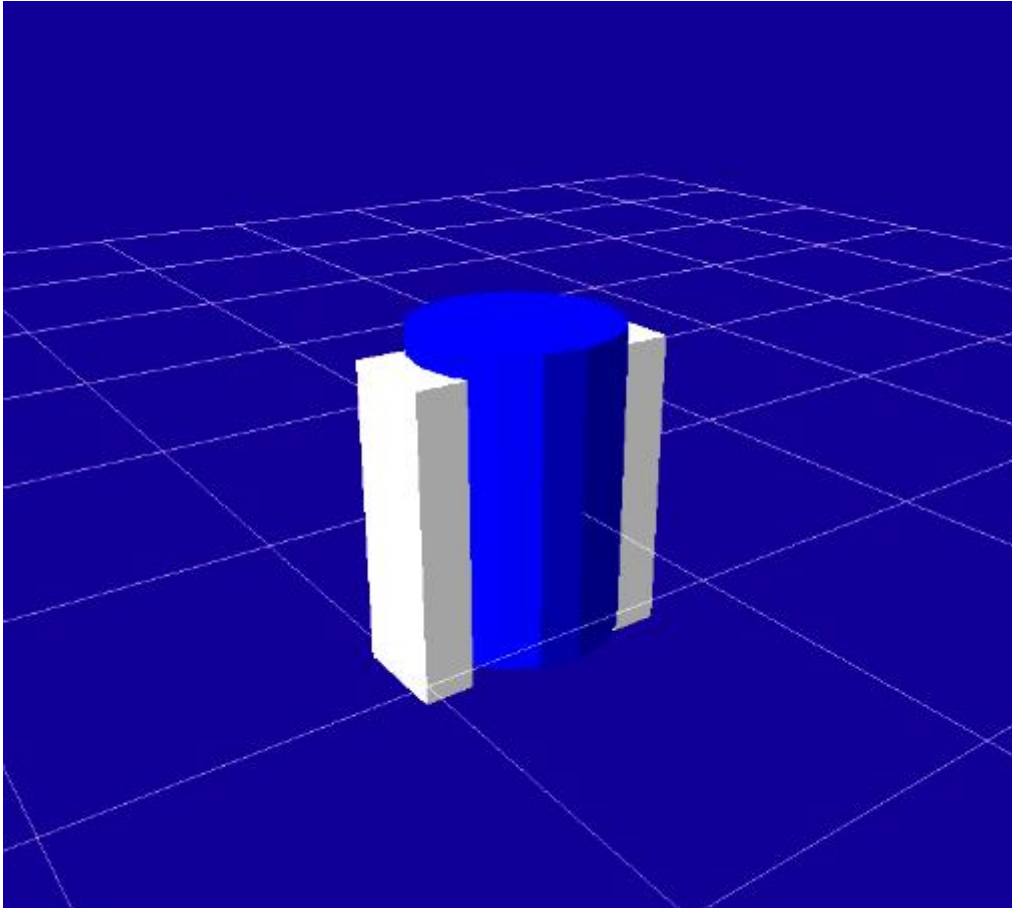
```
:wq
```

8) Input the command “**chmod +x simple\_demo.urdf**” and press Enter to grant the executable permissions to the saved **simple\_demo.urdf** file.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/urdf_demo/urdf$ chmod +x simple_demo.urdf
ubuntu@raspberrypi:~/hiwonder_ws/src/urdf_demo/urdf$
```

## 4.2 Load Robot Model

The model loaded by the above code appears is as follow:



Define a simple robot mode, with the main components and functionalities as follow:

- 1) Two materials are defined - blue and white.

```
<?xml version="1.0"?>
<robot name="materials">

  <material name="blue">
    <color rgba="0 0 0.8 1"/>
  </material>

  <material name="white">
    <color rgba="1 1 1 1"/>
  </material>
</robot>
```

- 2) Defined the base\_link using blue materials and cylinder geometry.

```
<link name="base_link">
  <visual>
    <geometry>
      <cylinder length="0.6" radius="0.2"/>
    </geometry>
    <material name="blue"/>
  </visual>
</link>
```

3) Defined the link of right leg and left leg using white material and box geometry.

```
<link name="right_leg">
  <visual>
    <geometry>
      <box size="0.6 0.1 0.2"/>
    </geometry>
    <origin rpy="0 1.57075 0" xyz="0 0 -0.3"/>
    <material name="white"/>
  </visual>
</link>
```

```
<link name="left_leg">
  <visual>
    <geometry>
      <box size="0.6 0.1 0.2"/>
    </geometry>
    <origin rpy="0 1.57075 0" xyz="0 0 -0.3"/>
    <material name="white"/>
  </visual>
</link>
```

4) Defined fixed rotational joints for the right leg and left leg, connecting them to the base link. The origin of the right leg joint is located at (0, -0.22, 0.25), and for the left leg, it's at (0, 0.22, 0.25). The leg boxes are rotated to stand upright.

```
<joint name="base_to_right_leg" type="fixed">
  <parent link="base_link"/>
  <child link="right_leg"/>
  <origin xyz="0 -0.22 0.25"/>
</joint>
```

```
<joint name="base_to_left_leg" type="fixed">  
  <parent link="base_link"/>  
  <child link="left_leg"/>  
  <origin xyz="0 0.22 0.25"/>  
</joint>
```

The main function is to define the materials, geometries, links and their connections or this simple robot.

By defining materials, links, visual components, joints, and other basic syntax, the definition of this robot is completed.